

고급머신러닝 보고서

YOLO

You Only Look Once

전자공학부 전자공학과
2020144005 김윤성
2024. 11. 05 제출

목차 INDEX

01 YOLO

- YOLO란?
- YOLO의 구조

02 실습 #1)

- YOLOv5 실습
- 코드 분석
- 실행 결과
- 결과 분석

03 실습 #2)

- YOLOv1 by keras
- 코드 분석
- 실행 결과
- 실행 비교
- 결과 분석

The background is a solid dark blue. In the top-left corner, there is a dark blue triangular shape. In the top-right corner, there is a light blue curved shape. In the bottom-right corner, there is a dark blue diagonal shape.

01

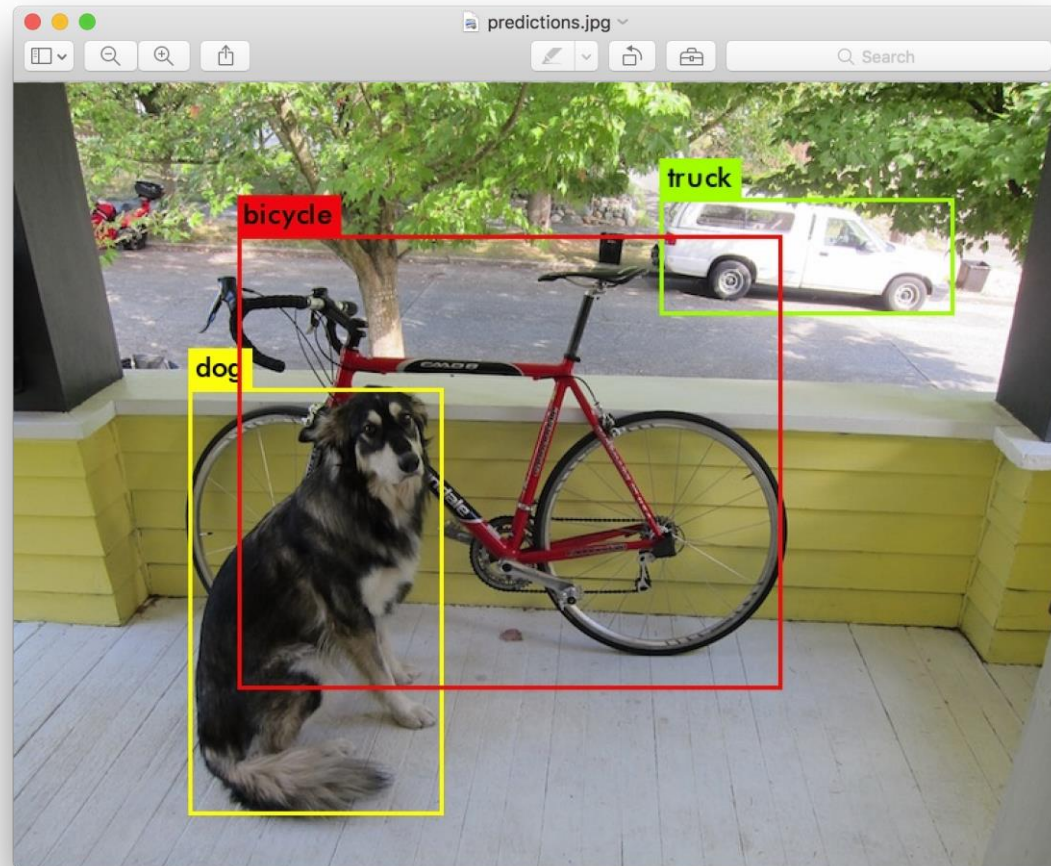
YOLO

You Only Look Once

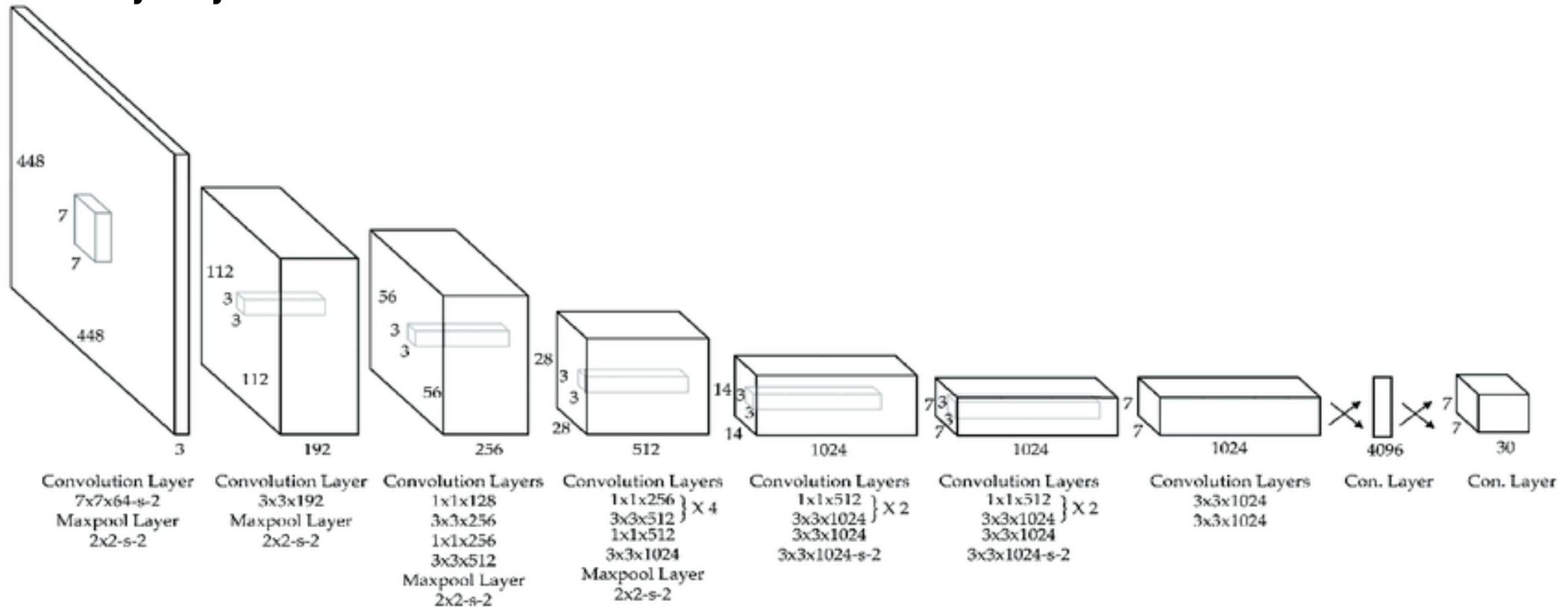
YOLO?

You Only Look Once

- R-CNN과 달리, 객체를 실시간으로 탐지하는 딥러닝 기반 알고리즘
- R-CNN같은 2-stage Detector 대비 빠른 속도, 높은 실시간 처리 성능, 약간 떨어지는 정확도
- 작거나 다른 객체와 겹쳐있는 객체에 대한 탐지성능 다소 떨어짐
- 이미지의 전역적 정보도 함께 고려
- 논문: [링크](#)



YOLO의 구조 You Only Look Once



인풋 레이어



Backbone



Convolution Layer



F.C Layer



Predict Tensor

02

실습 #1

YOLOv5 실습

실습 #1) YOLOv5 실습

- 여러 하이퍼 파라미터를 조정하며 최적화
- 학습에 사용할 data 개수를 자유롭게 설정하며 학습 진행
- 다양한 모델 사용해보며 다른 점 및 결과 분석(xlarge, large, middle등)
- 예측 결과를 R-CNN의 결과와 비교 분석

02 실습 #1 코드 분석



실행 환경: Google Colaboratory

```
from google.colab import drive # Connect between Google Drive and Colab
drive.mount('/content/drive')
```

```
!pwd
!ls /content/drive/MyDrive/YOLO_MNIST_data
```

```
# MNIST Unzipping
%cd /content/drive/MyDrive/YOLO_MNIST_data
!unzip -qq "MNIST.zip"
```

Colab과 Google Drive를 연동하고, 데이터의 압축을 해제한다.

Colab 링크 (클릭)

02 실습 #1 코드 분석

```
from glob import glob

img_list = glob("data/images/*.jpg")

from sklearn.model_selection import train_test_split

train_img_list, test_img_list = train_test_split(img_list, train_size=0.6)
train_img_list, val_img_list = train_test_split(train_img_list, train_size=0.6)

with open('data/train.txt', 'w') as f:
    f.write('\n'.join(train_img_list) + '\n')

with open('data/val.txt', 'w') as f:
    f.write('\n'.join(val_img_list) + '\n')

import yaml
import os

abs_train = os.path.abspath('data/train.txt')
abs_val = os.path.abspath('data/val.txt')
with open("data.yaml", 'r') as f:
    data = yaml.full_load(f)
data['train'] = abs_train
data['val'] = abs_val
data['nc'] = 10
data['names'] = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
with open("data.yaml", 'w') as f:
    yaml.dump(data, f)
```

파일 리스트를 불러오고, Set를 분할하고, yaml 파일을 수정한다.

02 실습 #1 코드 분석

```
!git clone https://github.com/ultralytics/yolov5.git
!pip install -r yolov5/requirements.txt
```

```
!WANDB_MODE=disabled python yolov5/train.py --img 256 --batch 16 --epoch 50 --data data.yaml --cfg yolov5/models/yolov5s.yaml
# wandb 관련 기능 비활성화
```

```
# YOLO로 생성된 Bounding Box를 PLT, OpenCV를 사용하여 밑에 표시
```

```
import matplotlib.pyplot as plt
import cv2

test_img_path = test_img_list[0]
!python yolov5/detect.py --weights yolov5/runs/train/exp2/weights/best.pt --img 256 --conf 0.5 --source {test_img_path} --save-
txt --save-conf --project yolov5/runs/detect --name result --exist-ok

result_img_path = 'yolov5/runs/detect/result/' + test_img_path.split('/')[-1] # YOLOv5 출력 이미지 경로

img = cv2.imread(result_img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 10))
plt.imshow(img)
plt.axis('off')
plt.show()
```

YOLOv5의 깃허브를 clone하여 YOLOv5를 실행한다.
이 환경에서는 WandB관련 오류가 발생하여 아예 비활성화시키고 실습을 진행했다.
matplotlib의 imshow()를 이용하여 이미지가 바로 표시되게 하였다.

02 실습 #1 실행 결과

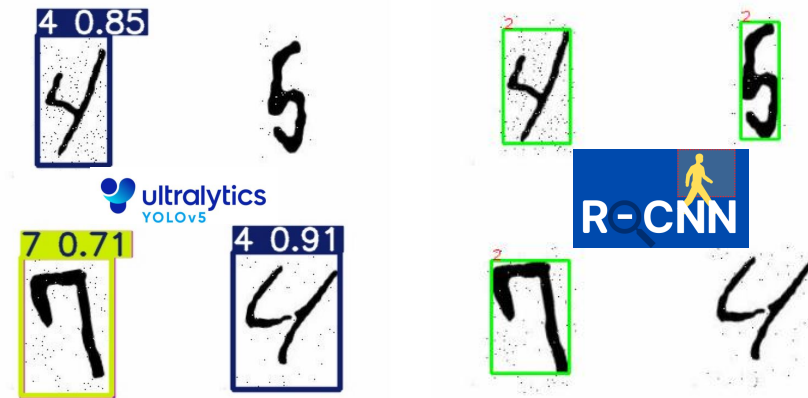


 **ultralytics**
YOLOv5



02 실습 #1 결과 분석

- 기존의 R-CNN과 비교하여, YOLOv5의 정확도와 시간이 모두 앞섬.
- YOLOv5와 같은 기성품들은 이미 Pre-Trained되어 성능이 더 뛰어난 것으로 추정됨.
- YOLOv5는 바운딩 박스의 예측 값과 신뢰도가 비교적 높음.
특히 각 바운딩 박스의 숫자 클래스와 신뢰도 점수가 표시되어 있어, 모델이 각 물체에 대해 얼마나 확신을 가지고 있는지 알 수 있음.
- YOLOv5는 바운딩박스가 숫자 주변의 공백을 약간 포함하는데 비해, R-CNN은 YOLOv5 대비 더 타이트한 바운딩박스를 가지고 있음.
이는 작은 객체를 더 잘 포착하는 R-CNN의 특성 때문일것으로 추정됨.
- YOLOv5와 R-CNN모두 숫자 4개 중 3개만 감지에 성공함.
- 반면, 1 stage detector인 YOLO의 특성상 R-CNN보다 훨씬 더 짧은 실행시간을 가졌음.
이는 YOLO가 실시간처리에 R-CNN보다 더 적합하다는것을 의미



03

실습 #2

YOLOv1 by keras

실습 #2) YOLOv1 구현

- keras를 활용하여 이론에서 배운 YOLOv1 구현
- 구현 성공 시, 아주 높은 과제 추가 점수 부여 -> 예측 결과 및 분석과 함께 보고서 작성
- 구현 실패한 경우에도 구현 과정과 방법을 설명하며 실패한 이유 분석 시,
- 추가 점수 부여 가능 (단, 코드만 붙인 경우 X)

03 실습 #2 코드 분석



실행 환경: 로컬 (Visual Studio Code)

```
import tensorflow as tf
from keras.layers import Input, Conv2D, Flatten, Dense, Reshape, Dropout, MaxPooling2D, BatchNormalization, LeakyReLU
from keras.models import Sequential, Model
import numpy as np
import os
import cv2
from sklearn.model_selection import train_test_split
import time
import matplotlib.pyplot as plt
import glob
```

필요한 모듈을 임포트한다.

파일의 경로와 일치하는 모든 파일을 찾기 위해 glob을 임포트했다.

```
image_dir = r'MNIST\\data\\images'
label_dir = r'MNIST\\data\\labels'

image_paths = glob.glob(os.path.join(image_dir, '*.jpg'))
label_paths = glob.glob(os.path.join(label_dir, '*.txt'))

image_paths.sort()
label_paths.sort()

train_images, test_images, train_labels, test_labels = train_test_split(image_paths, label_paths, test_size=0.3, random_state=42)
val_images, test_images, val_labels, test_labels = train_test_split(test_images, test_labels, test_size=0.5, random_state=42)
```

이미지와 레이블을 glob을 이용하여 불러오고, 경로를 정렬한 다음, 데이터를 나눈다.

03 실습 #2 코드 분석

```
def preprocess_image(image_path, target_size=(448, 448)):
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = cv2.resize(image, target_size)
    image = image / 255.0
    return image
```

데이터를 전처리하는 함수이다.

색도 RGB로 바꿔주고, 크기도 수정하는 역할을 한다.

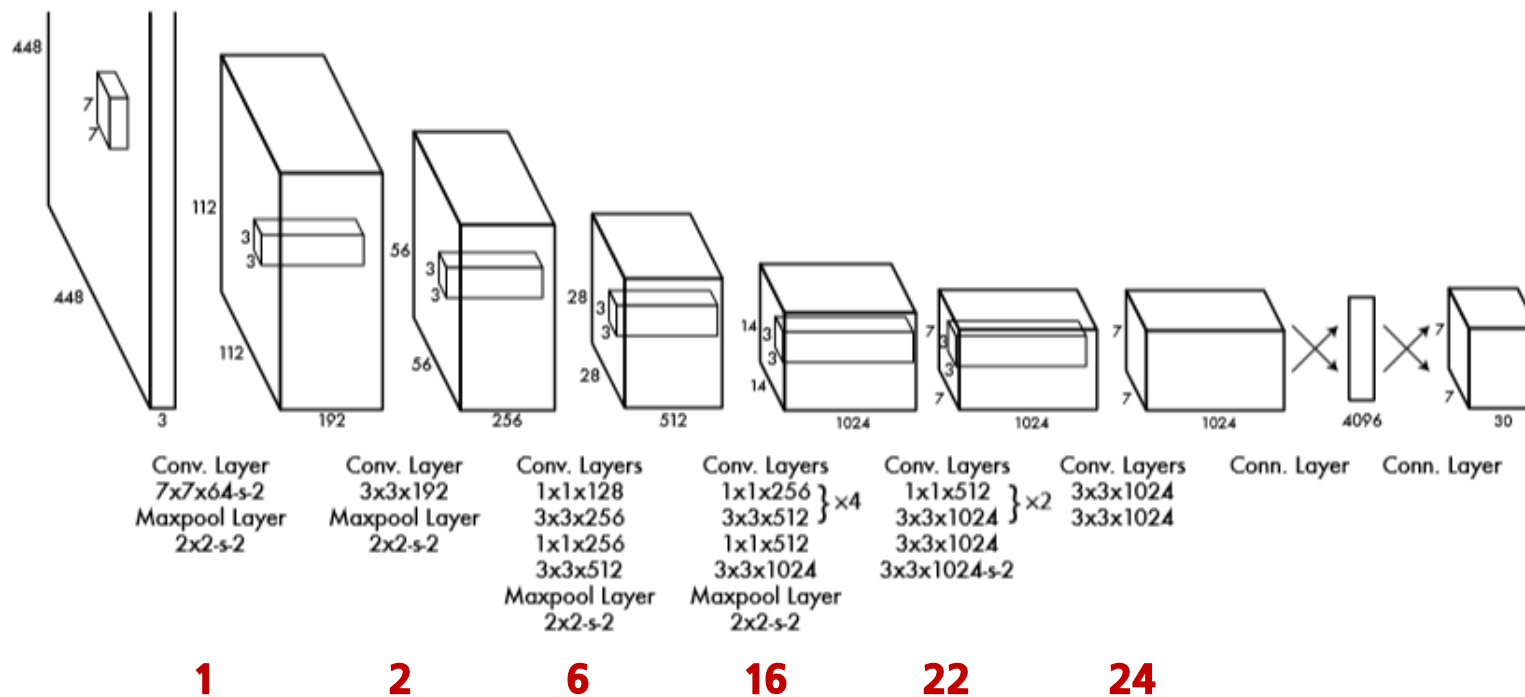
```
def load_labels(label_path):
    label_matrix = np.zeros((7, 7, 30)) # YOLOv1의 출력 크기와 동일하게 초기화
    with open(label_path, 'r', encoding='latin-1') as file:
        lines = file.readlines()
        for line in lines:
            parts = line.strip().split()
            class_id = int(parts[0])
            x_center, y_center, width, height = map(float, parts[1:])
            grid_x = int(x_center * 7)
            grid_y = int(y_center * 7)
            x_offset = x_center * 7 - grid_x
            y_offset = y_center * 7 - grid_y
            label_matrix[grid_y, grid_x, 0:4] = [x_offset, y_offset, width, height]
            label_matrix[grid_y, grid_x, 4] = 1.0 # 신뢰도
            label_matrix[grid_y, grid_x, 5 + class_id] = 1.0 # 클래스 원핫 인코딩
    return label_matrix
```

MNIST/data/labels에서 bounding box와 클래스 정보를 로드하는 함수이다.

03 실습 #2 코드 분석

```
def make_dataset(image_paths, label_paths):  
    images = [preprocess_image(img_path) for img_path in image_paths]  
    labels = [load_labels(lbl_path) for lbl_path in label_paths]  
    return np.array(images, dtype=np.float32), np.array(labels, dtype=np.float32)  
  
train_images_dataset, train_labels_dataset = make_dataset(train_images, train_labels)  
val_images_dataset, val_labels_dataset = make_dataset(val_images, val_labels)  
test_images_dataset, test_labels_dataset = make_dataset(test_images, test_labels)
```

데이터셋을 만들어주는 함수와, 그를 이용하여 Training, Validation, Test Set들을 생성하는 코드이다.



YOLOv1에는 총 24개의 Convolution Layer가 있다. 다음 부분에서는 이것을 구현할 것이다.

03 실습 #2 코드 분석

```
def yolo_v1(input_shape=(448, 448, 3)):
    inputs = Input(shape=input_shape)
    # Conv Layer 1
    x = Conv2D(64, (7, 7), strides=2, padding='same', use_bias=False, name='conv1')(inputs)
    x = BatchNormalization(name='bn1')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu1')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=2, padding='same', name='maxpool1')(x)
    # Conv Layer 2
    x = Conv2D(192, (3, 3), padding='same', use_bias=False, name='conv2')(x)
    x = BatchNormalization(name='bn2')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu2')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=2, padding='same', name='maxpool2')(x)
    # Conv Layers 3-6
    x = Conv2D(128, (1, 1), use_bias=False, name='conv3')(x)
    x = BatchNormalization(name='bn3')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu3')(x)
    x = Conv2D(256, (3, 3), padding='same', use_bias=False, name='conv4')(x)
    x = BatchNormalization(name='bn4')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu4')(x)
    x = Conv2D(256, (1, 1), use_bias=False, name='conv5')(x)
    x = BatchNormalization(name='bn5')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu5')(x)
    x = Conv2D(512, (3, 3), padding='same', use_bias=False, name='conv6')(x)
    x = BatchNormalization(name='bn6')(x)
    x = LeakyReLU(alpha=0.1, name='leaky_relu6')(x)
    x = MaxPooling2D(pool_size=(2, 2), strides=2, padding='same', name='maxpool3')(x)
```

1~6번 Convolution Layer의 코드이다.

논문에서는 YOLOv1의 Backbone으로 GoogleNet을 변형한 Darknet이라는 구조를 사용하였는데, 구글넷에서 Inception 모듈을 제거하고 1x1, 3x3필터를 반복적으로 사용하여 YOLO의 실시간 성능에 최적화된 구조이다.

GoogleNet과 Inception Module에 대한 설명은 합성곱신경망 보고서에 자세히 설명되어 있다.

03 실습 #2 코드 분석

```
# Conv Layers 7-14 (4 repeated blocks)
for i in range(4):
    x = Conv2D(256, (1, 1), use_bias=False, name=f'conv7_{i+1}')(x)
    x = BatchNormalization(name=f'bn7_{i+1}')(x)
    x = LeakyReLU(alpha=0.1, name=f'leaky_relu7_{i+1}')(x)
    x = Conv2D(512, (3, 3), padding='same', use_bias=False, name=f'conv8_{i+1}')(x)
    x = BatchNormalization(name=f'bn8_{i+1}')(x)
    x = LeakyReLU(alpha=0.1, name=f'leaky_relu8_{i+1}')(x)
# Conv Layer 15-16
x = Conv2D(512, (1, 1), use_bias=False, name='conv9')(x)
x = BatchNormalization(name='bn9')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu9')(x)
x = Conv2D(1024, (3, 3), padding='same', use_bias=False, name='conv10')(x)
x = BatchNormalization(name='bn10')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu10')(x)
x = MaxPooling2D(pool_size=(2, 2), strides=2, padding='same', name='maxpool4')(x)
# Conv Layers 17-20 (2 repeated blocks)
for i in range(2):
    x = Conv2D(512, (1, 1), use_bias=False, name=f'conv11_{i+1}')(x)
    x = BatchNormalization(name=f'bn11_{i+1}')(x)
    x = LeakyReLU(alpha=0.1, name=f'leaky_relu11_{i+1}')(x)
    x = Conv2D(1024, (3, 3), padding='same', use_bias=False, name=f'conv12_{i+1}')(x)
    x = BatchNormalization(name=f'bn12_{i+1}')(x)
    x = LeakyReLU(alpha=0.1, name=f'leaky_relu12_{i+1}')(x)
```

7~20번 Convolution Layer의 코드이다.

논문에서는 처음 20개의 컨볼루션 레이어가 Backbone으로 쓰인다고 설명하고 있으며, 따라서 여기까지가 Darknet의 코드라고 볼 수 있다.

03 실습 #2 코드 분석

```
# Conv Layers 21-22
x = Conv2D(1024, (3, 3), padding='same', use_bias=False, name='conv13')(x)
x = BatchNormalization(name='bn13')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu13')(x)
x = Conv2D(1024, (3, 3), strides=2, padding='same', use_bias=False, name='conv14')(x)
x = BatchNormalization(name='bn14')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu14')(x)
# Conv Layer 23-24
x = Conv2D(1024, (3, 3), use_bias=False, name='conv15')(x)
x = BatchNormalization(name='bn15')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu15')(x)
x = Conv2D(1024, (3, 3), padding='same', use_bias=False, name='conv16')(x)
x = BatchNormalization(name='bn16')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu16')(x)
# Fully Connected Layers
x = Flatten()(x)
x = Dense(4096, activation='linear', name='fc1')(x)
x = LeakyReLU(alpha=0.1, name='leaky_relu_fc1')(x)
x = Dropout(0.5)(x)
x = Dense(7 * 7 * 30, activation='linear', name='fc2')(x)
x = Reshape((7, 7, 30), name='output')(x)
model = Model(inputs=inputs, outputs=x, name="Darknet_YOLOv1")
return model
```

21~24번 Convolution Layer 및 Fully Connected Layers의 코드이다.

이 4개의 레이어들은 최종적으로 특징을 추출하고, 네트워크가 이미지의 작은 부분에서 특징을 더 세밀하게 추출할 수 있도록 하고, 보조적 Bounding Box에 대한 정보를 추출하며, 최종적인 Bounding Box의 위치와 크기를 조정하는 역할을 맡는다.

따라서, 이 4개의 레이어들은 객체 탐지의 세밀화, 지역적인 미세한 정보 추출 및 학습, 최종 출력 준비를 맡는다고 할 수 있다.

03 실습 #2 코드 분석

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad \textcircled{1}$$

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad \textcircled{2}$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \quad \textcircled{3}$$

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \quad \textcircled{4}$$

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad \textcircled{5}$$

YOLOv1의 손실함수는 위와 같은 구조이다.

예측된 Bounding Box의 크기와 정답을 비교해서 크기 및 위치 오류를 계산 (작은 박스가 위치변화에 더 민감하므로 w, h에 sqrt 사용)

예측된 Bounding Box에 객체가 있는지 없는지

예측된 Bounding Box안에 있는 객체가 무슨 클래스인지 (예측한 클래스가 맞는지)

이것들을 모두 더한 값을 손실함수로 사용한다.

03 실습 #2 코드 분석

```
@tf.function
def yolo_multitask_loss(y_true, y_pred):
    batch_size = tf.shape(y_true)[0]
    batch_loss = 0.0
    for i in tf.range(batch_size):
        y_true_unit = y_true[i]
        y_pred_unit = y_pred[i]
        for row in range(7):
            for col in range(7):
                # 바운딩 박스 정보
                bbox_true = y_true_unit[row, col, :4]
                bbox_pred = y_pred_unit[row, col, :4]
                # 신뢰도
                bbox_true_confidence = y_true_unit[row, col, 4]
                bbox_pred_confidence = y_pred_unit[row, col, 4]
                # 클래스 확률
                class_true = y_true_unit[row, col, 5:]
                class_pred = y_pred_unit[row, col, 5:]
                # The Localization Loss (좌표 차이의 제곱합)
                localization_err = tf.reduce_sum(tf.square(bbox_true - bbox_pred))
                # The Confidence Loss (차이의 제곱)
                confidence_err = tf.square(bbox_true_confidence - bbox_pred_confidence)
                # The Classification Loss (원-핫 인코딩된 클래스 확률 차이의 제곱합)
                classification_err = tf.reduce_sum(tf.square(class_true - class_pred))
                # Total Loss (가중치 적용)
                cell_loss = 5 * localization_err + confidence_err + classification_err
                batch_loss += cell_loss
    batch_loss /= tf.cast(batch_size, tf.float32)
    return batch_loss
```

1, 2

3, 4

5

YOLOv1을 실제로 구현할 때 가장 어려웠던 손실함수 코드이다.

아무리 생각해도 바운딩박스에 대한 정보가 가장 중요한 것 같아 바운딩박스의 오차에 가중치를 줌 주었다.

03 실습 #2 코드 분석

```
# 모델 컴파일
yolo_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005), loss=yolo_multitask_loss, metrics=['accuracy'])

# 모델 훈련 시작 시간 기록
start_time = time.time()

# 모델 훈련
yolo_model.fit(train_images_dataset, train_labels_dataset, epochs=EPOCHS, batch_size=BATCH_SIZE, validation_data=(val_images_dataset, val_labels_dataset))

# 모델 훈련 종료 시간 기록
end_time = time.time()

# 테스트 이미지에서 임의의 하나를 예측
random_idx = np.random.randint(0, test_images_dataset.shape[0])
test_image = test_images_dataset[random_idx]
test_label = test_labels_dataset[random_idx]

prediction = yolo_model.predict(np.expand_dims(test_image, axis=0))[0]
```

주석대로 모델을 컴파일하고, 결과를 그리기 위한 준비를 하는 단계이다.

03 실습 #2 코드 분석

```
# 결과 이미지 출력 및 바운딩 박스 시각화
plt.figure(figsize=(8, 8))
plt.imshow(test_image)
h, w, _ = test_image.shape
colors = ['r', 'g', 'b', 'c', 'm', 'y', 'orange', 'purple', 'pink', 'brown']
color_idx = 0
for i in range(7):
    for j in range(7):
        if prediction[i, j, 4] > 0.5: # 신뢰도가 0.5 이상인 경우만 시각화
            x_offset, y_offset, box_width, box_height = prediction[i, j, :4]
            x_center = (j + x_offset) / 7 * w
            y_center = (i + y_offset) / 7 * h
            box_w = box_width * w
            box_h = box_height * h
            box_x = int(x_center - box_w / 2)
            box_y = int(y_center - box_h / 2)
            rect = plt.Rectangle((box_x, box_y), box_w, box_h, linewidth=2, edgecolor=colors[color_idx % len(colors)], facecolor='none')
            plt.gca().add_patch(rect)
            class_id = np.argmax(prediction[i, j, 5:])
            confidence = prediction[i, j, 4] * 100
            plt.text(box_x, box_y, f'[{class_id}, {confidence:.2f}]%', bbox=dict(facecolor=colors[color_idx % len(colors)]), fontsize=12,
            color='white')
            color_idx += 1
plt.title('Test Image with Bounding Box')
plt.axis('off')

# 총 실행 시간 출력
print(f"총 실행 시간: {end_time - start_time:.2f} 초")

plt.show()
```

최종적으로 도출된 결과를 이미지로 나타내는 코드이다. 테스트 이미지중에서 무작위로 하나 뽑아서 이미지로 나타낸다.

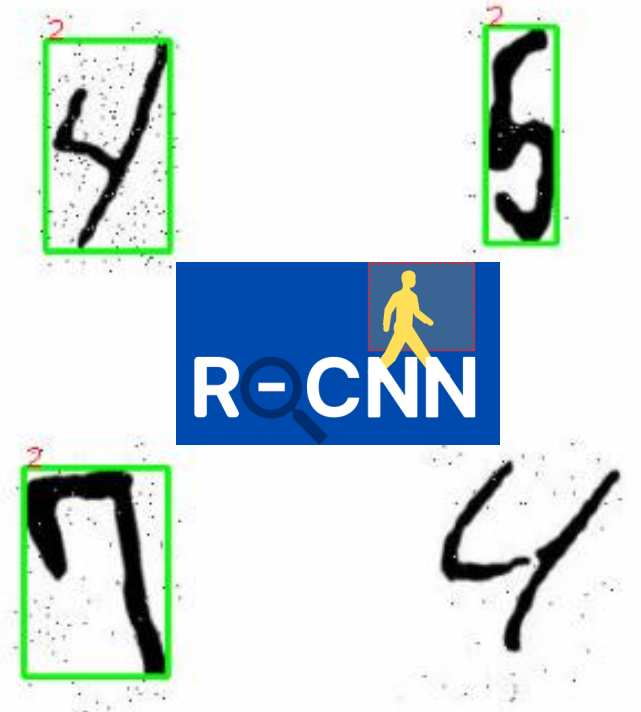
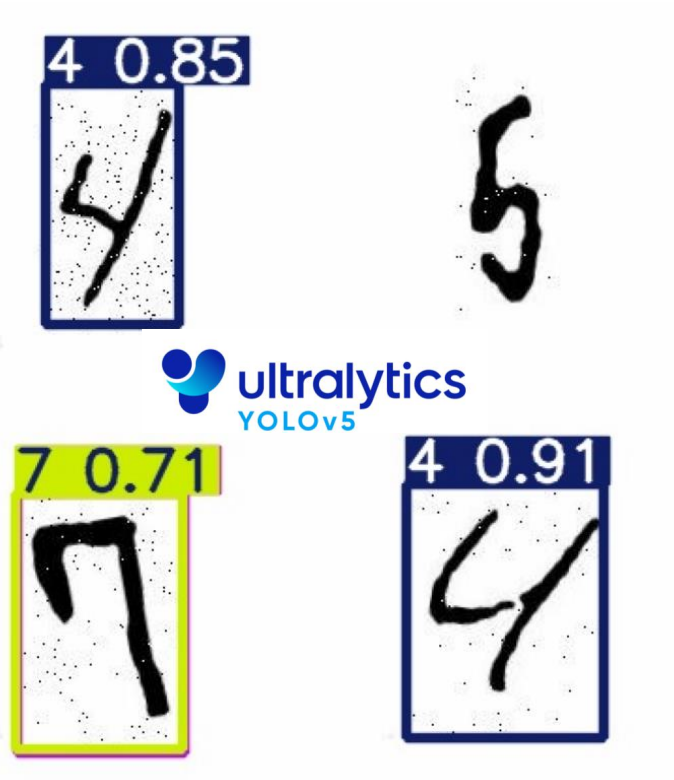
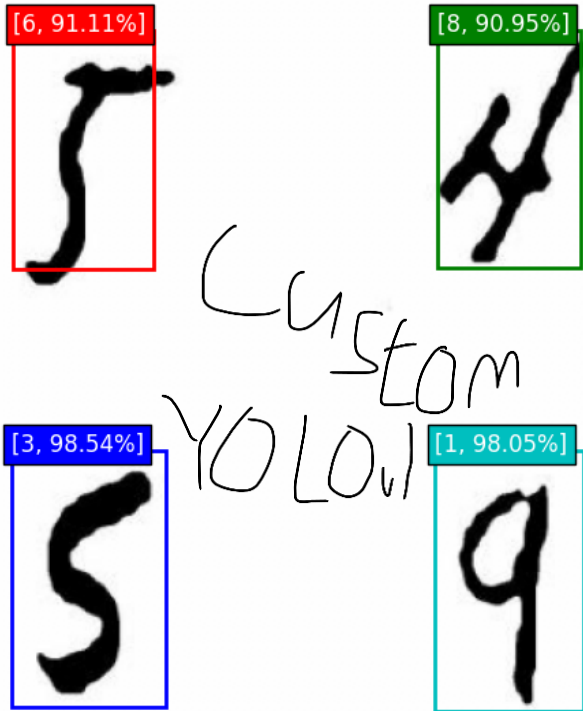
03 실습 #2 실행 결과

Test Image with Bounding Box



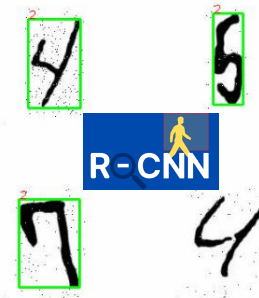
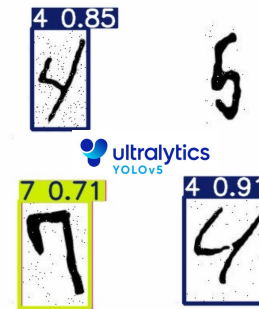
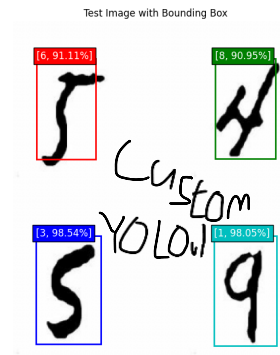
03 실습 #2 실행 비교: VS YOLOv5, R-CNN

Test Image with Bounding Box



03 실습 #2 결과 분석

- YOLOv1은 YOLOv5 대비 Bounding Box는 기깔나게 쳤으나, 분류 정확도가 안타까운것을 볼 수 있다.
- YOLOv1은 v5대비 Bounding Box가 타이트하다 못해 숫자가 박스에서 빠져나온것도 있는데, 이 또한 YOLOv5의 박스가 더 적합하다는 것을 보여줌
- YOLOv1은 클래스를 단 하나도 분류해내지 못함. 이는 학습률, epoch, 아니면 YOLOv1 아키텍처 고유의 문제일 가능성도 있음.
- YOLOv1은 epoch 50, batch size를 64로 했을 때 6498.12초가 걸림. 이는 R-CNN을 학습시킬 때에 비해 확실히 빠른 값
- Cross Stage Partial Darknet, Path Aggregation Network, Feature Pyramid Network, SiLU, Batch Normalization, DropBack 등등 발전된 구조를 가진 YOLOv5에 비해, 구현된 YOLOv1은 상대적으로 학습 시 안정성이나 분류 정확도가 매우 떨어짐



고급머신러닝 보고서
감사합니다